

A One-Month Programme for Continual-Learning Predictive-Coding Caps on GPT-2-Scale Transformers

Revised plan with controlled experiments and an executable implementation contract
(readable edition)

Ben Goertzel and Claude Fable

7 September 2026 – revised edition, rewritten for readability

Abstract

This document plans one month of work on continual-learning “caps”: small learning systems attached to a frozen GPT-2-scale transformer that read its internal states and write corrections back into it. The primary question is whether *routing* a learning signal to the place where a small intervention measurably improves the current prediction reduces damage to unrelated knowledge, while still allowing useful transfer through shared mechanisms, at comparable acquisition, memory, and compute. The cap is built first on the ordinary backpropagation-trained (BP) model. A separate controlled comparison then isolates three things that are easy to confuse: the effect of an error-based predictive-coding (ePC) representation of the base, the effect of “settled” error credit as opposed to gradient credit, and the effect of retrieval features derived from errors. The cap of this month (“v0”) is a PC-motivated gated activation memory; it does not yet contain its own predictive-coding inference process, and the document is careful not to claim otherwise. Three test beds are used: a constructed modular fixture in which the right routing is known by construction, a learned synthetic grammar with shared and private latent causes, and lifelong editing of complete answers on zsRE and CounterFact. A bounded development phase precedes a frozen confirmatory protocol. Directly measured update-order effects are the main interference diagnostic; Hessian commutators are an optional, explicitly local diagnostic. Re-distillation of the ePC base is conditional on finding a specific deficit, and never forces unrelated domains to be orthogonal. The appendices specify execution, metrics, memory updates, resource accounting, and reporting, including how to finish an experiment whose answer is negative.

Revision status. Revised 7 September 2026 from the original September plan, incorporating a technical review by OpenAI-6-Astra. This edition rewrites the exposition for a reader who knows machine learning but is new to predictive coding, model editing, and the methodology of controlled continual- learning experiments. The design, the numbers, and the implementation contract are unchanged from the 7 September revision. The document reports no new experimental results. Internal results quoted below are inherited from earlier reports and must be reproduced in the preflight before anything is built on them.

Contents

How to read this document	3
1 Purpose and summary	4
2 Background	5
3 Starting position	7
4 Programme structure	8
5 What makes a base cap-friendly	8
6 Thread 1: characterize, then conditionally regularize	10
7 Thread 2: the minimal cap and its highways	10
8 Measuring continual learning	11
9 Core experimental design and predictions	12
10 Schedule and decision points	13
11 Risks and traps	14
12 Outputs and relation to the wider programme	15
Appendix: contract for the coding agents	16
A Operating rules	16
B Environment, assets and budgets	17
C Stage specifications	19
D Metric definitions	22
E Stream specifications	28
F Cap v0 implementation contract	30
G Report template	34
H Revision map and interpretation boundaries	36

How to read this document

The document has two halves. The main body (Sections 1–12) explains what we are trying to find out, why the experiment is shaped the way it is, and what each result would and would not mean. It is written to be read from start to finish, and each technical idea is explained where it first appears. The appendix is a contract for the coding agents who will run the programme: it fixes numbers, procedures, and reporting formats. It is written to be looked up rather than read through, but every rule in it is accompanied by a short reason, so that an agent (or a person) who meets a rule out of context can see what it protects.

A recurring theme is the distinction between a *controlled contrast* and a *bundled comparison*. If two systems differ in three ways and one wins, we have learned that the bundle of three changes helps, not which change helped. Most of the structure of this plan exists to make sure that every comparison we draw conclusions from differs in exactly one thing.

The glossary below collects the terms and symbols used throughout. Readers who already know the area can skip it and return when needed.

Glossary and notation

Base

The frozen pretrained transformer. **BASE-BP** is the ordinary backpropagation-trained model; **BASE-EPC** is its conversion into an error-based predictive-coding network. D is the number of layers, d the width of the residual stream.

Residual stream

The vector $h_t^{(l)}$ carried through the network at layer l and token position t ; each transformer block adds its output to it.

Cap

The small learning system attached to the base. It reads residual-stream vectors and writes additive corrections back into them. Base weights never change.

Bank

One module of the cap, attached at one depth. There are three banks, at roughly one third, two thirds, and the full depth of the base.

Slot

One entry in a bank’s memory: a key (what to match against), a value (the correction to add), a radius (how close a query must be to fire the slot), and bounded metadata.

Key, query

The key is the stored retrieval feature of a slot; the query is the feature computed from the current input. A slot fires if the query is within the slot’s radius of its key.

Credit, credit direction

The signal that tells a bank which way to change a value so as to reduce the current loss. Two kinds are used: the adjoint and the settled error.

Adjoint

The gradient of the loss with respect to a hidden vector, obtained by reverse-mode automatic differentiation (backpropagation through the frozen base).

Settled error

In the ePC base, the error variable at a site after a fixed number of iterations of error inference conditioned on the target.

Delivery rule, highway

The rule deciding which banks receive credit for an item. The arms C0, C1, C2, CR, CO are delivery rules. “Highway” is the informal name for delivering credit to a bank far from the output.

Router

The component of C2 that chooses a bank by measuring where a small trial write improves the loss.

Arm

One complete configuration (base, delivery rule, key type, memory protocol) run on a stream.

Contrast

Two arms that differ in exactly one respect, so that a difference in results can be attributed to that respect.

Stream, realization, order

A stream is the ordered sequence of items to be learned. A realization is one sampled set of items; an order (permutation) is one ordering of that set. We use three realizations and five orders each.

Acquisition

Whether an item is learned at the time it is presented.

Retention

Whether an item is still known at the end of the stream.

Locality

Whether inputs unrelated to any learned item still receive the base’s answer.

Transfer

Whether learning one item helps (positive) or harms (negative) performance on others.

Development vs. confirmation

Development data may be used to tune and debug. Confirmation data are frozen in advance and touched once; results on them are the ones reported as findings.

Exposure-matched vs. resource-matched

Two ways of making arms comparable: processing the same items, or spending the same measured accelerator time.

1 Purpose and summary

The aim is to come out of the month with research results that are interesting to people outside our own group and that survive scrutiny. The object of study is a cap. A cap reads a frozen transformer’s internal states and writes corrections into its residual stream. Its job is to acquire new facts and tasks from a stream, retain the valid earlier ones, generalize to rephrasings, and leave unrelated behaviour intact. Freezing the base’s weights preserves the original model as a recoverable reference; note that it does not by itself preserve the *behaviour* of the model when the cap is switched on, since the cap’s writes change the forward computation. Locality therefore has to be measured, not assumed.

We keep the two ingredients of the original programme: characterize a transformer that has been converted to ePC form, and build a continually learning cap. The cap is built and evaluated on the BP model first, so that useful results do not depend on another successful conversion. The following are treated as three separate questions rather than one bundled comparison, because each has its own confounds:

1. **Routing.** Does delivering credit to a bank chosen by a measured intervention effect outperform delivering it to every bank, and also outperform choosing a bank at random with the same sparsity? Does it protect private mechanisms while still allowing transfer through shared ones? The random control is what separates “choosing well” from “writing less.”
2. **Representation.** Holding the credit rule (adjoints) and the key type fixed, does replacing the BP base with the converted ePC base change cap performance? This isolates the conversion itself.
3. **Credit and retrieval.** On the same ePC base, do settled errors improve on adjoints as a credit signal? Do error-derived keys help beyond a matched control that uses gradient-derived keys of the same dimension, once their query-time and storage costs are charged?

The guaranteed deliverable is one reproducible integrated report containing valid results, their limitations, and the failures along the way. A routing paper or an ePC substrate note follows only if the evidence warrants it. The multilevel predictive-coding architecture of the July design document remains the destination; this month establishes a dependable experimental base for it.

2 Background

2.1 Two ways of assigning credit

Credit assignment is the problem of deciding how a learning system should change after a wrong prediction. A gradient answers this for each parameter: it says how much the loss would change under a small nudge of that parameter. Backpropagation computes gradients by the chain rule, composing local derivatives along every path from the loss back to the parameter. Nothing in the network keeps a record of the errors it made; the gradient is computed, used, and discarded. But this is a convention, not a limitation: gradients can be recorded, analysed, and used to gate learning. Persistent error records are therefore not something only predictive coding can offer, and any advantage claimed for predictive coding has to be shown rather than assumed.

Predictive coding (PC) networks represent discrepancies explicitly. Each layer holds error units comparing what the layer predicted with what was actually inferred. A common PC learning cycle has two phases: first the activities are relaxed to reduce an energy (the total squared error, in the simplest version), and only then are weights updated from the resulting local signals. This “infer, then update” structure and the availability of local error signals make certain architectural choices natural, such as changing only the parts of the network that are surprised. Whether these choices reduce forgetting in practice depends on the model, the learning dynamics, the retrieval mechanism, and the data. One known difficulty with state-based PC implementations is that credit propagates slowly through depth. Our Coupling Dilemma note [3] motivates studying both this propagation problem and the interference problem together.

2.2 Error-based predictive coding

Error-based PC (ePC) reorganizes the inference phase. Instead of adjusting activities and reading off errors, it treats the errors themselves as the unknowns to be optimized, and reconstructs the activities forward from them. This avoids the exponential decay of signals through depth that slows state-based PC.

One detail matters for this programme and is easy to get wrong. The cited digital implementation [6] optimizes the errors using reverse-mode automatic differentiation; only the weight update afterwards is local. So a claim that ePC has “local plasticity” is not a claim that settling performs no backward computation. When we compare costs, the preflight therefore records forward evaluations, reverse evaluations, settling iterations, peak memory, and wall-clock time for every credit procedure, and we

do not describe settling as free. Nor does fast settling in the cited experiments guarantee that eight iterations suffice for the transformer used here; that is measured.

The internal causal-fibres report [5], as summarized in the original plan, reported a per-token KL divergence to the teacher near 3×10^{-5} (that is, the converted model’s next-token distribution is almost indistinguishable from the original’s), a cosine similarity above 0.998 between settled errors and adjoints (the two credit signals point in nearly the same direction), and error mass less concentrated in the last block than in the earlier R8/R9 runs. These are starting claims to reproduce, not results this revision has verified. The strong alignment makes a controlled credit comparison particularly informative: if the two signals nearly coincide, any difference in cap behaviour must come from something else, such as the geometry of the retrieval features or the behaviour of finite-iteration inference, and those are things we can test directly.

2.3 Caps, and what cap v0 is and is not

Adapters and LoRA [11] add small trainable pieces to a model (LoRA adds low-rank weight deltas). Lifelong model-editing methods go further and store corrections in a memory. GRACE [7] keeps a discrete key–value memory at one layer: each entry has a key, a value that replaces or shifts the hidden state, and a radius; a query fires an entry when it falls within the radius, and conflicts are handled by shrinking radii. WISE [8] instead keeps a parametric side memory derived from a feed-forward value matrix, routes queries between the main and side memories, and shards and merges knowledge across edits. Their memory representations and resource costs differ, so a fair comparison must measure those costs rather than count “edits.”

Cap v0, the cap of this month, is a **PC-motivated gated activation memory**: it stores local corrections in radius-gated slots at three depths and receives credit that is either an adjoint or a settled ePC error. It does *not* contain a separate cap-level energy with an infer-then-update state dynamics of its own. This matters for how results are described. Calling an output residual “explicit,” or weighting an update by a loss statistic, is not sufficient to establish a predictive-coding mechanism inside the cap. A future genuine PC cap must specify its own energy, its inference variables, its convergence procedure, and its plasticity rule. None of the routing results below depend on adopting that stronger label, and the report will not use it.

2.4 Continual learning

Continual learning means acquiring information from a stream, one item after another, with bounded retained state and without going back to retrain on everything. We measure immediate acquisition (did the item get learned when it was presented), final retention (is it still known at the end), paraphrase performance (is it known under rephrasing), locality (are unrelated inputs unaffected), forward and backward transfer (did earlier learning help or hurt later items, and vice versa), and the ability to acquire new items late in the stream, since some systems lose plasticity as they fill up [13].

Two cautions shape the metrics. First, a model that learns nothing has zero forgetting, so retention and order invariance are always reported together with acquisition; a low forgetting number on its own means nothing. Second, when a stream contains an explicit correction of an earlier fact, the two answers cannot both be right; corrections are evaluated against the latest valid fact, not against every answer ever supplied.

2.5 Error highways and order effects

An error highway supplies credit to write sites far from the output, so that banks at early depths can learn. The programme tests three delivery rules: full delivery to all banks, measured selection of

one bank, and sparse random selection of one bank. The measured router asks a simple question: where does a small trial write improve the current loss? That is an *intervention effect*. It is not automatically an identification of the latent mechanism that produced the error. A late bank can almost always shift the output logits in a helpful direction, whatever caused the mistake, so “the probe helps here” and “the error originated here” are different statements. The connection between measured routing and the causal routing proposed in [1] is therefore a hypothesis, tested on controlled fixtures where the causal structure is known by construction.

The same care applies to how interference is measured. Let U_i denote the complete update the learner performs for item i , including allocation, radius changes, and eviction, not just a gradient step. Order dependence is observed directly: starting from the same cloned state, apply U_j after U_i and U_i after U_j , and compare the two resulting predictions. For the special case of smooth gradient steps $U_i(\theta) = \theta - \eta g_i(\theta)$, expanding to second order gives

$$U_j(U_i(\theta)) - U_i(U_j(\theta)) = \eta^2 \{H_j g_i - H_i g_j\} + O(\eta^3), \quad (1)$$

where g_i and H_i are the gradient and Hessian of item i ’s loss. This expression is worth staring at: it involves the gradients, not only the Hessians. So commuting Hessians alone do not imply commuting updates. The smallest possible example makes the point. For $L_1 = \theta^2/2$ and $L_2 = (\theta - 1)^2/2$, both Hessians equal one and so commute trivially, yet $g_1 = \theta$ and $g_2 = \theta - 1$, and the order difference is exactly η^2 : each step contracts toward a different minimum, and it matters which contraction comes second. Hessian commutators, the object studied in the Shadow Calculus note [4], describe a more restricted linearized phenomenon; Appendix D.9 states exactly what they can and cannot tell us. Two further points keep the interpretation honest: zero order dependence does not imply an absence of sharing, since shared parameters can support commuting updates; and the Shadow Calculus is motivation for the measurements, not a substitute for checking the hypotheses in the implemented system.

3 Starting position

3.1 Assets

The month does not start from nothing. The original plan identifies the following assets. Stage S0 confirms their paths, versions, licences where relevant, and reproducibility before anything relies on them.

- The production ePC checkpoint and its BP teacher, with the 5×10^7 -token distillation pipeline that produced it.
- Causal-fibres v0.3, including joint block diagonalization (a method for finding a subspace that several sets of vectors share) and the supplied-support plasticity sandbox (a test bed in which the cap is told exactly which inputs an edit should affect).
- The comcrit Hessian-vector-product and quadratic-control code, used for the optional curvature diagnostics.
- The six-layer synthetic-grammar fixture used for the R8/R9 and E1–E2 runs.
- RelaLeap experiment harnesses, where reusable.
- FabricPC/JAX, only if the existing checkpoint requires it and it is already suitable for these experiments. Optimizing that framework is not a deliverable of this month.

3.2 Lessons from earlier runs

The reported R8/R9 runs reached low distillation energy while concentrating credit in the final block and collapsing the internal rank of the representation; energy descent alone did not certify

a usable model. The August preflight reportedly failed feedforward fidelity despite deterministic, energy-monotone runs. The CMCP comparison was reportedly confounded by unequal distillation mass between arms. These experiences motivate four habits used throughout: functional preflights (does the model compute what it should, before anything else), geometry diagnostics (is the internal representation intact), matched exposure (arms see the same data), and explicit resource accounting (arms are charged for what they consume).

One further lesson is about how failures are read. A failed experiment does not by itself establish that the implementation, rather than the idea, was at fault. Each failure is classified, with evidence, as one of four things: a reproducible implementation defect, a resource limitation, an unmet assumption, or a valid negative result. Development may repair a demonstrated defect; confirmatory results are not re-tuned until a hypothesis passes.

4 Programme structure

Thread 1 concerns the base: measure it, and conditionally test one targeted modification. Thread 2 concerns the cap: build it and identify what routing contributes, on the BP base. The threads share data manifests, metrics, and a budget, but an unavailable ePC arm does not stop the BP programme.

	Thread 1: base	Thread 2: cap
Week 1	Reproduce checkpoint claims; initial report card	Validate fixtures and baselines; calibrate; profile 100–300 edits
Week 2	Diagnose a specific substrate deficit; optional S6 decision	BP screening; matched routing controls; freeze confirmatory protocol
Week 3	Controlled representation/credit comparison; optional regularized pair	Confirmatory routing and editing runs; actual update-order probes
Week 4	Targeted substrate ablations if affordable	Replication, failure audits, integrated report; two-day buffer

Table 1: A shared development-to-confirmation workflow. A second cap level, domain adaptation, and broad all-arm Hessian sweeps are deferred to a later month.

The order of work follows one principle: build the measurement before the thing to be measured, and build the control arm before the arm we hope will win. Week 1 is almost entirely measurement and calibration; the interesting comparisons run in week 3 under a protocol frozen at the end of week 2.

5 What makes a base cap-friendly

The substrate report card is a set of six measured properties of a base. They play three different roles, and the roles must not be confused. A *validity requirement* says whether a base computes what it is supposed to; failing it makes results on that base uninterpretable. A *resource eligibility* criterion says whether a base can be run within budget. A *scientific descriptor* is something we measure because we hypothesize it matters, and whose relation to cap performance is itself under test. Only the first two can exclude a base from a comparison.

Some explanation of each property and of why its interpretation is hedged.

P1, fidelity asks whether the converted base still computes the teacher’s function. KL divergence per token is the standard way to compare two next-token distributions; a mean of 10^{-3} nats says the two models’ distributions are very close on average. But an average over language-model tokens does

Property	Measurement	Role and interpretation
P1 Fidelity	Teacher KL, feedforward and unclamped; edit-prompt agreement	$\leq 10^{-3}$ nats on held-out LM tokens for an eligible matched-substrate claim; report tails and edit-set discrepancies
P2 Geometry	Effective rank and linear probes by layer	Report ratios; 0.9 rank ratio and 2-point probe difference are diagnostic alerts, not universal necessity theorems
P3 Localization	Per-example participation; dataset layer coverage	Sparse individual fields must not hide systematic single-layer concentration
P4 Structure	Subspace overlap, private/shared transfer and intervention tests	Domain PCA is descriptive; useful shared structure is not penalized indiscriminately
P5 Write locality	Edit improvement and collateral at matched success	Report gated and unconditional interventions separately, including failures to reach target success
P6 Cost and informativeness	Settling residuals, steps, runtime; error-loss association	Eight steps are the nominal credit budget; 16 is an optional resource screen; correlation is not calibrated uncertainty

Table 2: The substrate report card. Exact measurements and boundary cases are in Appendix D.

not guarantee agreement on the particular edit prompts we care about, so we also check agreement there, and we report the tail of the distribution, not only its mean.

P2, geometry guards against the R8/R9 failure: a base can match the teacher’s outputs while its internal representation has collapsed to a few directions, leaving a cap nothing to read. Effective rank measures how many directions carry variance; a linear probe measures whether simple information (part of speech, or a latent label on the synthetic grammar) is still linearly readable. Both are *alerts*, not exclusion criteria: lower effective rank can be useful compression rather than damage, and a probe drop may or may not matter to the cap. An alert triggers a fixed diagnostic, not the silent removal of a working model.

P3, localization asks where the error mass lives. A cap benefits from error fields that are sparse per example. But per-example sparsity can hide a systematic problem, namely that every example’s mass sits in the same single layer; so we also report layer coverage across the dataset.

P4, structure asks whether the error vectors from different domains or mechanisms occupy different subspaces. Principal-component overlap between domains is descriptive: nearly orthogonal private subspaces may make selective writes easier, but overlap does not make routing impossible, because distributions, nonlinear features, and context can still distinguish inputs. And shared subspaces can be beneficial; they are what transfer runs through. So no domain-overlap number is penalized indiscriminately. The decisive evidence is the effect on retained and transferred knowledge, measured by interventions.

P5, write locality measures one particular kind of edit direction: how much a write along the base’s own credit direction improves the edited prompt, and how much collateral damage the same write does elsewhere. It is one substrate advantage among several possible ones, not the whole story.

P6, cost and informativeness records what settling costs and whether the error magnitude is associated with the loss. A correlation is an association; it is not a calibrated uncertainty estimate and is not called one.

Two general cautions apply to the whole card. Error norms and PCA geometry depend on coordinates: on layer scales and on the convention used for error precision. The wrapper records those conventions, and every comparison uses the same observation and intervention protocol. And rankings of P4/P5 across two or three bases are descriptive case studies. Repeated cap runs on one fixed checkpoint do not create independent substrate replicates; with two or three checkpoints there is no basis for a statistical law relating properties to performance.

6 Thread 1: characterize, then conditionally regularize

The first job is measurement. Score three credit signals on matched prompts: BP adjoints, ePC adjoints (the converted base is still differentiable, so its gradient is available too), and ePC settled errors. Audit the energy function, the clamping convention (how the target is imposed during settling), the differentiation graph (what reverse-mode passes actually run), and the actual compute. Use the existing ePC checkpoint even if no further training turns out to be affordable.

The second job is conditional. Stage S6, a re-distillation with an added regularizer, is launched only if development identifies a reproducible deficit and a tractable intervention for it. Two families are permitted:

- If the deficit is *localization* (error fields too diffuse), the intervention is a positive participation penalty, which pushes each example’s error mass toward fewer cells, together with a separate dataset-level coverage term. The coverage term exists because the cheapest way to make every field sparse is to park all credit in one block, which is the R8/R9 failure; the coverage term discourages that.
- If the deficit is *private-mechanism interference* (learning one private mechanism disturbs another), the intervention is a synthetic, explicitly labelled private-overlap term that penalizes overlap only between mechanisms the fixture designates as unrelated, while preserving a supplied shared subspace. Broad orthogonalization of all domains is not the default, because it would destroy the shared structure that transfer needs.

Whichever intervention is chosen, it is evaluated as a *matched continuation pair*: a plain continuation and its regularized counterpart both start from the same production checkpoint and see the same training-token sequence, optimizer schedule, and KD mass. Comparing an additionally trained checkpoint only with the older production checkpoint would not isolate the regularizer, because extra training by itself changes the model. Both arms of the pair must fit the conditional budget. Appendix C specifies the objectives and their limits.

7 Thread 2: the minimal cap and its highways

The July design [2] proposes episodic, contextual, and schema levels, novelty quarantine, several plasticity timescales, and replay. This month builds one memory level with three possible write depths; the extra levels are deferred.

Each bank reads a normalized residual-stream feature at the current prediction position and writes an additive vector back at that position and depth. A slot contains a retrieval key, a correction value, a radius, bounded metadata, and usage statistics. Memory is bounded in *bytes*, including keys and metadata, not in slot counts. The reason is a capacity confound: three banks of 2,048 slots hold three times as much as one bank of 2,048, and keys that include error features are twice as wide as plain keys, so counting slots would let an arm win by having more memory. The v0 allocation rule is specified explicitly in Appendix F rather than described as an exact copy of GRACE.

The primary arms, and the confound each exists to remove:

C0: last-depth, matched memory.

Only the last bank is writable, with the same total memory ceiling as the three-bank arms. This is a depth-access control: it tells us whether access to earlier write depths helps at all. It is not an implementation of local-PC credit propagation.

C1: full delivery.

All three banks are eligible on every item. A common aggregate normalized write budget is divided among them, so that C1 does not simply write three times as much.

C2: measured routing.

Small signed loss-improving probes select one bank per round, or abstain. All bases use this same bank-level rule.

CR: sparse random routing.

One bank is selected from a distribution fixed in advance using development-only C2 delivery frequencies, with a matched nominal write budget. If C2 beats C1 but not CR, the benefit came from writing sparsely, not from choosing where. Realized usage and abstention differences are reported, because C2 may abstain on a round while CR’s schedule is fixed in advance, so the two arms can differ in how much they actually write.

CO: supplied routing.

On the constructed modular fixture, the permitted banks are supplied to the router. This is a mechanism positive control and an upper reference for that fixture: it shows what correct routing could achieve there. It is not a real-text baseline.

Two refinements. A smaller 2,048-slot last-bank arm is optional and is labelled as capacity-unmatched. And if CR and C2 end up with materially different realized write counts, a development-calibrated thinning control (CR with its writes thinned to C2’s realized rate) is reported as a diagnostic; unobserved test labels or future C2 decisions are never used to set CR’s schedule, because that would leak information from the test stream into the control.

A subtle point about keys. The primary keys are computed from the current *edited* forward pass, that is, with the cap’s earlier banks already writing. This means learning in an earlier bank can move the retrieval features seen by a later bank, even though the base’s weights are frozen. This *retrieval drift* is logged, and a selected ablation reads keys from a separate cap-disabled pass instead, charging the cost of the extra pass. Error-based keys are a later, separately controlled ablation.

Finally, the scope of what changes. Highways change how credit is delivered to the cap. The cap’s writes do change the forward computation when the cap is enabled; that is how it has any effect. Base parameters stay frozen, and no long-range generative connections are added to the base this month.

8 Measuring continual learning

Three test beds are used. They answer different questions, and no one of them could replace the others.

Constructed modular control. A small fixture with explicit shared and private modules and masked, known causal paths, so that the correct routing is known by construction. It answers two questions that cannot be answered on a real transformer: can correct routing help at all, and does our measurement code recover an identifiable case when one exists? It is a check on the apparatus and on the idea’s coherence. It is not presented as evidence that a transformer’s mechanisms are equally localized.

Learned synthetic grammar. The existing learned transformer receives tasks with shared and private latent switches. Task context is observable in the input, so identical available information

never requires incompatible labels (a stream that asked the model to give two different answers to identical inputs would be unlearnable by anyone, and would measure nothing). Attribution of a latent cause to a depth can span several depths on a learned model, and ambiguous tracing outcomes remain in the report rather than being tidied away. Held-out combinations of switches test transfer, including whether selective learning preserves the shared mechanisms.

Sequence-level editing. zsRE and CounterFact are run as separate streams, with complete answer strings, aliases, paraphrases, and near-neighbour locality prompts. The ceilings are 3,000 zsRE edits and 1,000 CounterFact edits per stream realization; the confirmatory prefix length is chosen from throughput measurements before testing. Edits are learned and evaluated as complete generated answers, not single tokens: a first token can be correct while the rest of the answer is wrong, and a single-token benchmark cannot see that. A single-token subset may serve as a development smoke test, labelled as such. Appendix E fixes the autoregressive training and free-generation evaluation protocol.

Baselines. B0 is the frozen model, which sets the floor; B1 is sequential LoRA with no protection against forgetting; B3 is LoRA with a bounded reservoir replay buffer, the standard practical defence; B4 is GRACE, adapted from its authors’ implementation and the closest published relative of the cap. B2 (LoRA with EWC [12]) and B5 (WISE) are optional. All run locally under the shared harness; published scores at other scales are never substituted. A joint-training reference on the synthetic fixture estimates the performance attainable with all data at once, without being assumed to be a mathematical ceiling.

Metrics. Immediate and retained acquisition, paraphrase retention, forward and backward transfer, locality, language-model drift, learning late in the stream, memory occupancy, evictions, update norms, runtime, and per-example order dependence. Order dependence is measured directly by the cloned-state probes of Section 2.5, which capture positive as well as harmful changes. Near-neighbour prompts, compositions of learned facts, and explicit corrections receive separate challenge evaluations. One principle runs through all of this: similar aggregate scores do not imply similar predictions, so the report keeps the per-item information needed to see which items were lost.

9 Core experimental design and predictions

9.1 Controlled contrasts

After development screening, the required routing contrasts on the BP base are C2 versus C1 and C2 versus CR. C0 identifies the effect of access to earlier depths; B3 and B4 provide practical reference comparisons. CO is confined to the constructed fixture. Combinations that answer no question are not run, and there is no full factorial grid.

The substrate comparison changes one thing at a time. All arms use C1 as the fixed delivery rule, R-h keys (plain hidden-state keys), and the same memory and update protocol:

Arm	Base	Credit	Contrast identified
SB	BP	Adjoint	Reference
SE-A	ePC	Adjoint	Conversion/representation under a common credit rule
SE-E	ePC	Settled error	Credit procedure on the same converted base

Reading the table as two steps: SB to SE-A changes only the base, so a difference is attributable to the conversion; SE-A to SE-E changes only the credit rule, so a difference is attributable to settled credit. C2 repeats these contrasts only if budget permits, and a favourable routing result does not retroactively make C2 the primary substrate arm; that choice is fixed in advance. Error-derived keys (R-e) are compared on the ePC base with plain keys (R-h) and with adjoint-feature keys of the same dimension (R-g); BP R-g is also measured. The R-g control exists so that a gain from R-e cannot be explained simply by keys having twice as many coordinates. Regularized bases from S6 are exploratory unless they are independently preregistered before their confirmatory cap evaluations.

9.2 Predictions and interpretations

Each prediction is stated with what a positive and a negative result would mean, because the meaning of a null is where interpretation most often goes wrong.

PR-A: depth access.

At matched total memory and nominal write budget, earlier write sites may improve acquisition or transfer relative to C0. A null means no demonstrated depth-access advantage in this setting, not the failure of all local-PC approaches.

PR-B: routing.

C2 improves retained paraphrase accuracy relative to C1 and to CR, with acquisition and locality noninferior as specified in Appendix D.11. Reduced direct update-order harm and reduced private-mechanism damage are supporting outcomes. Failure to outperform CR blocks attributing an advantage to measured selection rather than to sparse writing.

PR-C: causal agreement.

On the constructed fixture, C2’s routing precision and recall against the supplied permissions target 0.8. Below 0.6 is poor agreement; 0.6–0.8 is partial. Uncertainty intervals are reported. Failure here is a scientific outcome once the fixture and oracle controls pass, not an implementation failure. Tracing on the learned transformer is evaluated separately and is not called ground truth.

PR-D: substrate and credit.

SE-A versus SB and SE-E versus SE-A test separate hypotheses. An error-key claim additionally requires its feature control (R-g) and its resource accounting. Results are conditional on fidelity and on the particular cap tested.

PR-E: property association.

The P4/P5 scores and retention may covary across checkpoints. With two or three checkpoints this is descriptive, not a validated predictive law.

For the intended account of shared and private learning, both beneficial transfer and reduced harm are needed; reduced harm alone could be achieved by learning less. No Hessian statistic on its own constitutes success.

10 Schedule and decision points

The schedule follows a development-then-confirmation shape. In development, we may look at results and change things. At the freeze (stage S4) we commit, in a machine-readable manifest, every remaining choice: configuration, thresholds, which comparisons will be made, in what order the endpoints will be read, and the analysis code. After the freeze, confirmatory data are touched once. The purpose is to prevent the most common way that experiments fool their authors: adjusting the analysis after seeing which version of it looks best.

Week 1: development and preflight.

S0 builds the smallest validated harness; S1 scores the existing bases; S2 calibrates shared

numerics, validates the baseline adaptations, and profiles 100–300 edits to learn what things cost. Calibration data are disjoint from the confirmatory streams.

D1: validity and feasibility.

Record which bases are eligible for which claims. A functional BP programme continues even if ePC fails fidelity or exceeds its resource allocation. A geometry alert causes an explicit diagnostic, not an undocumented change to a number.

Week 2: mechanism screening.

S3 exercises C0/C1/C2/CR/CO on the fixtures and on short BP editing streams. S4 freezes configuration, outcome thresholds, pairing, stream lengths, and budget before confirmatory test execution. S6 is authorized in the run manifest only if its specific deficit and its matched continuation pair are affordable.

D2: implementation or science.

A failed correctness control stops the affected mechanism for a documented repair. A correctly implemented router that performs poorly proceeds to a bounded negative-result evaluation. Beating LoRA is not a validity gate; it is a result.

Week 3: confirmation.

Run the frozen BP contrasts and the S5 substrate comparisons. S7 computes actual update-order probes on committed checkpoints, using independent data. Optional experiments cannot consume the budget reserved for completing the core.

D3: scope.

Audit coverage, resource use, and uncertainty. If something must be dropped, drop the optional error-key, regularization, or Hessian extensions before reducing a paired core comparison. Never switch the primary endpoint to whichever metric looks favourable.

Week 4: verification and report.

S8 completes selected replications and diagnostic ablations, audits failures, and produces one integrated report. Two working days are preserved for overruns. A second cap level and domain adaptation are next month’s work.

11 Risks and traps

Misattributed advantages. Sparse writing, larger memory, useful retrieval features, and extra compute can each improve results on their own. The matched controls and one-change-at-a-time contrasts exist so that the report can say which of these explanations the evidence supports and which it rules out.

Retrieval drift. Earlier writes move the features that later banks key on. We log key displacement, retrieval changes, duplicate allocations, and their causes, and compare against stable (cap-disabled) keys when the drift looks material.

Unlearnable ambiguity. Conflicting labels without any observable context cannot all be retained by any method. The synthetic generator therefore exposes task context, and true corrections follow an explicit revision protocol and are excluded from claims about invariance to item order.

Metrics without acquisition. Low forgetting, small commutators, or stable outputs can all result from weak learning. Acquisition and retained accuracy are always reported together. Correctly rejected updates and abstentions are outcomes, recorded with codes, not hidden as exceptions.

Resource assumptions. Intervention probes, backward evaluations, settling, and per-token-prefix updates can dominate cost. Measured throughput, not hope, determines stream length; nominal “steps” are not comparable units of computation across methods.

Premature certainty. A negative P5 result does not exhaust the substrate hypotheses, and an attractive PCA pattern does not establish causal modularity. Every conclusion in the report states its assumptions and the range over which it was tested.

12 Outputs and relation to the wider programme

The deliverable is one integrated report containing the substrate report card, the controlled routing results, the resource frontier (performance against measured cost), the direct order-effect diagnostics, all deviations, and all failures. The source, pinned environment, manifests, stream generators, cap implementation, and baseline adaptations are released where licensing permits. A standalone routing paper or ePC note may be extracted from the report when the evidence supports it; three separate papers are not a requirement of the month.

The longer-term aim is unchanged: a transformer-plus-cap system whose internal structure supports reliable continual learning and, eventually, symbolic read/write interfaces in the Hyperon stack. This month’s contribution is an operational account of which representations, credit mechanisms, and retrieval policies actually help, and by how much, at what cost.

Appendix: contract for the coding agents

The main text and this contract describe one experiment. If they contradict each other, that is a specification defect: record it and resolve it during development, rather than silently choosing whichever reading produces a run. At the S4 freeze, the committed configuration and manifest instantiate every remaining development choice. Each rule below carries a short reason, so that a rule met out of context can be applied in the spirit intended.

A Operating rules

1. **Read and instantiate.** Read the complete plan. S0 records checkpoints, datasets, hardware, exact dependencies, and missing assets. A missing optional asset does not block unrelated valid work. *Reason: most implementation failures in earlier runs came from acting on part of a specification.*
2. **Three classes of decision.** Correctness controls test code; resource limits delimit feasible execution; scientific outcomes test hypotheses. A scientific null never becomes a code failure merely because it is undesirable. *Reason: treating a disappointing result as a bug invites tuning until the result pleases.*
3. **Bounded development.** Stages S0–S3 use development-only examples and at most the calibration candidates listed in this contract. Log every attempted configuration, including failed ones. No confirmatory prompts or labels are used to choose configurations. *Reason: a configuration chosen by looking at test data is a result measured on its own training data.*
4. **Freeze before confirmation.** S4 commits a machine-readable manifest containing all numerical choices, hashes, randomization, the complete list of comparisons, the order in which endpoints are read, stopping rules, and analysis code. A later bugfix creates a new version: retain the affected old results, and rerun all affected paired arms if affordable; otherwise mark the comparison incomplete. *Reason: a comparison whose two sides ran on different code is not a comparison.*
5. **Correctness failure.** Stop only the affected run or mechanism. Record the failed invariant, at most three diagnoses, and the cheapest test that separates them. A demonstrated implementation bug may be repaired within the development budget. Scientific thresholds cannot be relaxed to manufacture a pass. Unresolved changes to task scope go to the human lead. *Reason: halting everything on one failure wastes the month; relaxing a threshold destroys the meaning of passing it.*
6. **Shared measurements.** Use the same harness, evaluation prompts, tokenization, answer normalization, stream order, and revision metadata for every arm in a comparison. Missing metrics are explicit and are never replaced by arm-specific approximations. *Reason: two arms scored by different code can differ for reasons that have nothing to do with the arms.*
7. **Honest resource matching.** Report the exposure-matched and resource-matched views separately (Appendix B). An equal count of updates is not an equal count of FLOPs. Include replay, routing probes, line searches, and retrieval in the accounting. *Reason: a method that wins by spending three times the compute has not been shown to be a better method.*
8. **No silent fallback.** Exceptions and unexpected nonfinite values are logged with stack traces and the affected items. Mathematically undefined ratios receive a structured **undefined** value together with their operands; these are not NaNs in the computation graph. Ordinary rejection, eviction, unreachable targets, and ambiguous items have explicit outcome codes. *Reason: a fallback that changes a number without saying so turns a failure into a fabricated result.*
9. **Reproducibility.** Replaying the same configuration, input stream, and initial random state twice must reproduce discrete decisions and metrics. Use deterministic kernels for the small

controls. Where numerical nondeterminism is unavoidable, identify the kernel, require a metric tolerance of 10^{-6} relative or 10^{-8} absolute, and report decision disagreements rather than concealing them. Dropout is disabled in base evaluation and in cap probes. *Reason: if two runs of the same thing differ, no difference between two different things can be trusted.*

10. **Frozen base.** Hash base parameters and persistent buffers before and after each run. Cap-off predictions must reproduce the same wrapper’s uncapped reference. BP/ePC fidelity is a KL comparison between different checkpoints, not a demand for bit-exact identity. *Reason: a cap that quietly modifies its base is no longer a cap.*
11. **Baseline fidelity.** Prefer validated adaptations of the authors’ code. Record commits, local changes, and a small parity test. If a reimplementaion is necessary, label it and test it against an available reference on shared inputs before making comparative claims. *Reason: beating a broken baseline is not a finding.*
12. **Complete reporting.** File a stage report including all planned and completed arms, time spent, failures, exclusions, and interpretation. Stop at the resource ceiling and preserve checkpoints. Never continue sampling seeds because a significance threshold was almost reached. *Reason: adding seeds until a threshold is crossed guarantees crossing it eventually, whether or not the effect is real.*

B Environment, assets and budgets

Bases. BASE-BP is the teacher; BASE-EPC is its existing conversion. They share architecture, tokenizer, and interface. If the pair is smaller than GPT-2-small (124M parameters, $D = 12$, $d = 768$), the substrate experiment uses that matched pair, and a separate BP-only GPT-2-small experiment tests whether the cap transfers to a more competent model. Do not imply that a small synthetic-grammar model provides a competent factual editing substrate. If a matched pair has no usable natural-language capability, its substrate comparison remains synthetic and is labelled accordingly. EPC-CONT and EPC-REG name the matched continuation pair from S6. *Reason: the substrate comparison is only valid between models of the same architecture; the transfer experiment is only meaningful on a model that knows some facts.*

Software and paths. Python 3.11 and the existing production environment where possible. PyTorch handles the cap and the harness; JAX is used through the base wrapper only if the checkpoint requires it. Cross-framework gradients must be explicitly implemented and tested: copying arrays between frameworks does not establish a differentiable path. Optional HVP diagnostics may be unavailable for a wrapper without second derivatives, which does not invalidate the behavioural experiment. Record code commits, environment lock, checkpoint hashes, and tokenizer revisions. Results live under `results/stage/arm/base/realization/permutation/`; each directory contains configuration, metrics, decisions, cost ledger, and a resumable checkpoint.

Randomization. Three independent stream realizations (seeds 0, 1, 2), each with five order permutations. A realization changes the sampled examples and the within-task orders, not merely an unused seed in deterministic code. Match realization and permutation across arms. Synthetic task permutations use a committed balanced schedule over eight tasks. For editing, realizations are independently sampled subject-disjoint subsets where feasible, with five shuffled orders using seeds 100–104; unavoidable overlaps are logged. Cap initialization and random router or replay seeds are named separately. All conclusions condition on the chosen checkpoints; there are no independent base-training replicates unless S0 finds them and adds an explicit experiment. *Reason: variation*

across realizations is the only source of statistical uncertainty about data; variation across orders measures order sensitivity; the two must be separable.

Memory ceiling. For standard R-h keys, define a reference ceiling

$$B_{\text{cap}} = 6144(8d + 128) \text{ bytes.}$$

This is what 6,144 slots with FP32 keys and values of width d occupy, plus a 128-byte allowance per slot for bounded metadata; implementations use the actual allocated size, including alignment, if larger. Variable-length prompt strings may not be hidden outside this accounting. At $d = 768$ the reference is about 36.75 MiB. Divide the same total ceiling equally across the three banks of three-bank arms; give all of it to C0’s last bank. With d_k key coordinates, the permissible slot count per bank is

$$S_m = \left\lfloor \frac{B_m}{4d_k + 4d + b_{\text{meta}}} \right\rfloor,$$

so that wider keys (R-e, R-g with $d_k = 2d$) buy fewer slots. Report allocated and occupied bytes, key and value dimensions, occupancy, and training peak memory separately. All persistent statistics, replay tokens, optimizer state, and correction indices are counted for their respective arms. B3 has a replay ceiling of 5% of seen edits and B_{cap} bytes, whichever binds first; this does not make its total state equal to a cap’s, so report the actual total. B4 is run with the same cap-memory ceiling when its reference implementation supports a budget; disclose the eviction adaptation. *Reason: bytes are the only unit in which a three-bank memory, a one-bank memory, a wide-key memory, and a replay buffer can be compared.*

Two resource views. An *exposure-matched* run processes the same complete input stream and uses the same cap write-distance budget, while measuring its actual cost. A *resource-matched* run additionally receives the same measured accelerator-time ceiling and ends at the last completed item. Plot retained performance against both consumed time and completed items; evaluate the longest completed common prefix across the compared runs. Do not compare different acquired prefixes as though they were the same learning task. A run halted mid-item rolls that item back for endpoint evaluation, but its consumed compute is charged. For a joint “comparable compute” claim, measured update time must be within 20%, or the resource-matched view must support the claim; otherwise report the tradeoff. *Reason: exposure matching answers “which method learns more from the same data”; resource matching answers “which method learns more for the same money”; both questions are legitimate and they can have different answers.*

Cost ledger. Count full and partial forward evaluations, reverse/VJP evaluations, settle iterations, token-prefix microsteps, router probes, memory search time, wall time, accelerator time, and peak memory. Report query costs separately from learning costs. Charge any cap-disabled feature pass, self-label gradient, or settling to every query that uses it. Measure complete runs; operation counts explain cost, they do not substitute for runtime measurements.

Stage ceilings. The total envelope is 154 A100-equivalent hours, with the hardware conversion based on a measured shared workload. These are allocations, not predicted runtimes.

Stage	S0	S1	S2	S3	S4	S5	S6	S7	S8	Total
Hours	8	12	12	16	36	24	20	10	16	154

S6 is optional and its allocation covers *both* continuation arms. Its unused allocation may fund core replication under a logged budget amendment; the total never increases silently. S4 freezes the largest affordable common edit prefix from $\{300, 1000, 3000\}$ for zsRE and $\{300, 1000\}$ for CounterFact, and the training-sequence count from $\{256, 1024, 10000\}$ per synthetic task. Selection uses only pilot throughput with 25% headroom, and applies to every compared arm. If the smallest complete core comparison is unaffordable, report a feasibility result rather than an underpowered imitation of the planned sweep. *Reason: a stream length chosen after seeing results is a tuned parameter; a stream length chosen from throughput is not.*

C Stage specifications

The dependency graph is S0 to S1/S2; S2 to S3; S1/S3 to the S4 freeze; S4 to the confirmatory BP runs, S5, and S7. S6 is conditional on S1/S3 and independently budgeted. S8 closes the programme. Development and confirmation use disjoint manifests. Independent jobs may run concurrently when their inputs are ready.

S0: Harness and invariant tests (week 1)

Implement the base wrappers, bounded memory, stream readers, answer metrics, cloning of *complete* learner state (including RNG and counters), and the cost ledger. Keep HVP infrastructure off the critical path.

Required tests, each with a known answer: a uniform rank- r singular spectrum has effective rank r ; unequal nonzero singular values can give effective rank below the algebraic rank; uniform positive mass has participation ratio equal to its number of cells; zero mass has its own outcome code; orthogonal/identical subspaces have overlap 0/1; unchanged predictions have zero JS divergence; the scalar quadratic of Section 2.5 shows an update-order displacement of η^2 despite a zero Hessian commutator. Test cap-off identity, memory accounting, transactional rollback, and complete-state cloning before any experimental run. *Reason: a metric that has never been checked against a case with a known answer is a guess with a decimal point.*

S1: Substrate report card (week 1; parallel with S2)

Use a disjoint held-out LM set H of 2×10^6 tokens for P1 and sampled subsets for the other properties. Score BP adjoints, ePC adjoints, and ePC settled fields. Record the exact output clamp/loss and local energy conventions. Check prediction agreement on development editing prompts in addition to LM KL. Test the energy gradient at 8 and 64 iterations; 64 is a reference horizon, not an assertion of convergence. Apply D.1–D.6, including all undefined and unreachable cases. Output a validity/eligibility table and a descriptive report.

S2: Shared calibration, baseline validation, throughput (week 1)

Calibrate before running baselines. Development data: 300 edits with paraphrases and at least 1,000 unrelated or near-neighbour prompts, all disjoint from test.

Radius calibration. For each layer and read variant, choose the largest initial radius on a 20-quantile grid of observed key distances whose unrelated false-fire rate is at most 1%; break ties by paraphrase coverage. If no positive radius qualifies, use exact-key retrieval for the pilot and report that limitation. Store all radius candidates and their results. *Reason: the radius trades generalization (larger fires on paraphrases) against locality (larger fires on unrelated prompts); a 1% false-fire rate fixes the locality side so that arms are compared at equal risk.*

Update numerics. The cap starts with a normalized aggregate step budget $A = 0.1$ per token round, probe size $\epsilon = 0.01$, and at most five rounds. At most the three values $A \in \{0.03, 0.1, 0.3\}$ are screened. Freeze one shared choice using immediate acquisition subject to the same locality criterion; do not tune a separate learning rule per confirmatory cap arm. The radius calibration is common to C0/C1/C2/CR within a base and read variant. If development shows that these candidates cannot function, complete the diagnostic before proposing a changed protocol; do not consume confirmatory examples.

Baselines. B1/B3 start with LoRA rank 8 on the query and value projections of all layers, Adam at learning rate 10^{-4} , 10 optimizer steps per complete edit, and one epoch per synthetic task; test learning rates $\{3 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}\}$ under the same development allocation. B3 samples a reservoir and mixes one replay item per new item per step where available; charge all tokens and optimizer state. B4 follows its validated reference optimization, initially 100 value steps, with its layer and radius selected on development data. A radius is shared numerically with the cap only if the normalization and distance conventions also match. B2/B5 are optional and require their own validated manifests; there is no hidden untuned comparator labelled state of the art.

Throughput. Profile 100–300 complete edits, including query evaluation, each cap routing rule, and the ePC credit procedure. Produce operation counts and throughput projections. Exclude compile/warmup from steady-state latency but report those costs separately. S4’s run scope follows these measurements.

S3: Mechanism screening on BP (week 2)

Run the F.5 controls, then C0/C1/C2/CR/CO on the constructed fixture and C0/C1/C2/CR on the learned grammar. Use one development realization and two orders; add short editing runs for baseline and implementation checks. Log routes, accepted writes, collisions, retrieval drift, evictions, and rejected updates. CO must solve the planted modular fixture to its specified attainable level; C2 is *not* required to identify the oracle bank as an implementation gate, because whether it does is the scientific question. If C2 fails, determine whether probes, retrieval, capacity, or the causal hypothesis explains the result. A valid negative result proceeds to the frozen bounded evaluation. Estimate CR’s fixed bank probabilities from this stage only.

S4: Freeze and confirmatory BP comparisons (late week 2 – week 3)

Commit the full manifest before any test access. Required routing arms are C1, C2, and CR on the learned grammar and on the affordable sequence-editing streams; include B3 and B4 as practical editing comparators. C0 is required on the grammar and on the initial editing scope; its long extension is lower priority. B0/B1 are reference curves wherever already available under identical streams. Use three realizations and five permutations, all paired. The endpoint order, margins, and uncertainty method are those of D.11. The run schedule is fixed independently of observed advantages. Output acquisition/retention/locality tables and cost frontiers. Record incomplete comparisons without imputing missing runs.

S5: Controlled substrate comparison (week 3)

For eligible matched bases, run SB, SE-A, SE-E with C1, R-h keys, matched streams, byte ceilings, and write policy. Eight settled iterations are the nominal SE-E procedure; other horizons are development or separately registered diagnostic arms. If the natural-language comparison is ineligible, perform the matched synthetic comparison and keep BP-only editing as a separate result. C2 repetition and R-e/R-g key contrasts are optional. Every R-e contrast includes same-dimensional R-g on ePC;

BP R-g is measured if the cross-base feature claim is made. The paired SE-A/SE-E comparison remains interpretable even if ePC fails the teacher-fidelity criterion, but then it cannot establish matched-fidelity superiority over BP; state the distinction explicitly.

S6: Conditional matched re-distillation (weeks 2–3)

Write a deficit statement and the intended intervention before starting. Both continuation arms begin at the same production checkpoint and see the same token sequence and KD weights. KD mass is $\sum_t w_{\text{KD},t} n_t$, with n_t the training tokens at step t . Match optimizer schedule and total exposures as well as this scalar. Reserve cost for both arms and their evaluation before committing.

Localization intervention. Let $m_{l,t}$ be the D.3 mass, N the number of cells, and q_l the batch-aggregated share of mass in layer l .

$$L_{\text{part}} = \mathbb{E}[\text{PR}(m)/N], \quad (2)$$

$$L_{\text{cov}} = \text{KL}(q^{\text{ref}} \| q), \quad (3)$$

$$L = L_{\text{KD}} + \lambda_p L_{\text{part}} + \lambda_c L_{\text{cov}}. \quad (4)$$

The sign of L_{part} is *positive*: participation ratio is high when mass is diffuse, so minimizing it concentrates mass. q^{ref} is a fixed, development-derived layer-coverage target obtained from useful adjoint interventions; floor its entries at 0.01 before renormalizing. Add 10^{-8} before forming shares. These terms discourage diffuse individual fields and loss of useful layer coverage, respectively; they do not establish causal modularity. Average the normalized PR only over examples with total mass above 10^{-12} ; if none qualify, set that batch term to zero and log an all-zero-field event, so that the objective is never undefined. Zero or near-zero fields are monitored separately against acquisition and unregularized mass; they are not counted as ideal localization. *Reason: a field can be made “sparse” by making it vanish, which is not what anyone wants; the masking and the coverage term close that loophole.*

Private-overlap intervention. Only on a fixture with supplied shared/private structure,

$$L = L_{\text{KD}} + \lambda_s L_{\text{private}} + \lambda_h L_{\text{shared}}.$$

Fix a shared basis Q_l from development data before training. Form private residuals $(I - Q_l Q_l^\top)e$ and penalize the D.4 overlap only for pairs the fixture designates as unrelated. Preserve the supplied shared component using $L_{\text{shared}} = \mathbb{E}\|Q_l^\top(e - e^{\text{start}})\|^2$, with the starting checkpoint’s field frozen on the same examples. Verify shared transfer behaviourally; feature preservation is only a proxy. If the private residual rank is insufficient, mark that term unsupported rather than fabricating a PCA basis. No domain-only claim licenses applying this separation loss to all domain pairs.

Common rules. Normalize active loss terms by fixed nonzero development reference magnitudes. Start active regularizer weights at 0.01; permit one development-only comparison at one quarter of that value within the S6 ceiling. The selected objective and weights freeze before the final held-out evaluation. Do not combine both families this month without a separate matched ablation. Gradients through adjoint or error features require higher-order or unrolled differentiation: test that the regularizer actually changes parameter gradients, since silently detaching the error feature would nullify the intervention while appearing to run. Log costs. Output both checkpoints, data/optimizer parity, P1–P6, and the fidelity cost of the regularizer.

S7: Actual order effects and selected diagnostics (weeks 3–4)

Use saved checkpoints from S4/S5 and independent probes. D.9 requires cloned-state pair reversals for selected shared and private edit pairs. Compute per-example order disagreement from the existing

permutation sweeps. Optional HVP diagnostics run only on a small fixed parameter block with fixed topology; no broad all-arm sweep is required. PR-E is a descriptive checkpoint table.

S8: Targeted ablations and integrated report (week 4)

Prioritize a fixed small list: cap-disabled stable keys versus edited keys; fixed-byte memory at half and double the reference; optional difficulty weighting versus none; R-e versus R-g if those arms exist. Choose the list from development failure mechanisms and commit it before its test runs. Use three realizations and two permutations for exploratory ablations, labelled accordingly. Finish the core paired analyses first. Do not build a second cap level. Output the report, reproduction commands, and all manifests, including unscheduled and failed work.

D Metric definitions

All metrics are computed by the shared harness. Accuracy-like scores are fractions, so a “two-point margin” means 0.02. Use natural logarithms throughout. Store numerators, denominators, sample counts, and exclusion reasons with every ratio. For schema purposes **undefined**, **unreachable**, and **not-supported** are distinct outcome codes and are distinct from an unexpected floating-point NaN. *Reason: a ratio without its operands cannot be audited; an undefined value that is silently replaced by zero or by a large number changes an average.*

D.1 Fidelity (P1)

Compute $\text{KL}(p_T||p_S)$ between teacher and student next-token distributions, averaged over H at the same teacher-forced positions: first with no settling, then without a target clamp after eight iterations. An eligible matched-fidelity comparison requires both means $\leq 10^{-3}$ nats per token. Report also the 95th and 99th percentile and the maximum finite token KL, base task accuracy, and argmax agreement on the actual edit-prompt pool; a mean LM threshold is not a guarantee of fidelity on edits. Select the immutable edit stream using the BP teacher only, then report each other base’s initially-correct fraction without changing its stream, and include the common initially-incorrect subset as a secondary comparison.

The wrapper specifies exactly what “unclamped” means. If removing the output loss and initializing errors to zero makes this check identical to the feedforward computation, report it as a consistency check, not as evidence of nontrivial inference. The cap-disabled identity control compares each base with its own reference forward computation.

D.2 Geometry and non-collapse (P2)

For each layer, centre hidden features from 4,096 held-out token positions. With singular values s_i and $p_i = s_i^2 / \sum_j s_j^2$, the effective rank is

$$r_{\text{eff}} = \exp\left(-\sum_{i:p_i>0} p_i \log p_i\right),$$

the exponential of the entropy of the normalized spectrum: it counts how many directions carry variance, weighting each by how much. If all singular values vanish, report a zero matrix and rank zero; do not divide by zero. A rank- r matrix has $r_{\text{eff}} = r$ only when its nonzero singular values are equal. Report per-layer ratios to the teacher, full spectra, and sampling seeds. A ratio below 0.9 triggers a diagnostic alert.

For natural language, use a fixed linear part-of-speech probe with at least 20,000 training and 5,000 held-out tagged tokens. For the synthetic fixture use the supplied latent/context labels instead of inventing POS labels. Probe training is disjoint from evaluation and matched across bases. More than two accuracy points of degradation is an alert; whether it affects cap utility is tested, not assumed. Record normalization conventions before comparing geometry.

D.3 Error localization and coverage (P3)

For a common teacher-forced sequence, BP uses the adjoint of the summed next-token loss at each hidden site. ePC uses the error variable after the specified finite inference procedure, with the same observed targets. In its own declared coordinates, let $m_{l,t} = \|e_{l,t}\|^2$ (or the squared adjoint norm). For nonzero mass,

$$\text{PR}(m) = \frac{(\sum_{l,t} m_{l,t})^2}{\sum_{l,t} m_{l,t}^2}, \quad \text{nPR} = \text{PR} / N.$$

PR is the “effective number of cells” carrying mass: it equals N for a uniform field and 1 for a field concentrated in one cell. Report the mean and distribution over 1,000 sequences, the final-block share, and the fraction of cells above 1% of the sequence’s maximum mass. The latter is an *active fraction*; it decreases with sparsity. Zero fields are counted separately. Report raw norms and a layer-scale-normalized diagnostic; neither is coordinate invariant.

Also aggregate layer shares over the dataset, stratified by private/shared mechanism where known. A useful sparse field may cover different layers on different inputs. A final-block share above 0.4 is an alert inherited from the original plan, not a universal theory of useful credit. S6 uses acquisition and coverage checks to prevent a low participation score from being mistaken for success.

D.4 Separability and useful sharing (P4)

For each domain or mechanism and each layer, sample 8,192 error vectors from a disjoint set, centre them, and extract an orthonormal top- r basis. Set $r = 16$ only if both datasets and their retained residuals support that rank; otherwise report an insufficient-rank case or a separately labelled common lower-rank result. For equal-rank bases U_a, U_b ,

$$O_{ab} = r^{-1} \|U_a^\top U_b\|_F^2 \in [0, 1],$$

which is the mean squared cosine of the principal angles between the two subspaces: 0 when they are orthogonal, 1 when they coincide. Report the matrix, captured variance, eigenvalue gaps, and resampling stability. Compare domain PCA descriptively. On the synthetic fixture, separately report private/private overlap for independent mechanisms, shared-component retention, and held-out compositional transfer. The joint block diagonalizer may provide an additional description, not an independently supplied causal ground truth. Any claim of private/shared structure requires intervention and transfer checks.

D.5 Credit locality and retrieval behaviour (P5)

Use 200 development edit prompts Q and 200 unrelated or near-neighbour prompts U , with held-out replicas for confirmatory measurements. At each write depth, form the candidate unit descent direction under the declared credit rule. A bounded geometric search (F.3) seeks a 50% reduction of the current token loss. Record the achieved improvement, the required normalized write norm, and any inability to reach the target; compare directions at matched achieved success where possible.

For a write v_q , report the unconditional collateral

$$C_q = |U|^{-1} \sum_{u \in U} \text{KL}(p_u(0) \| p_u(v_q)),$$

obtained by applying that same write at the corresponding prediction position of every u . Also measure the actual gated cap on U , including firing rate and complete-answer changes. These measure different phenomena: harm if a gate misfires, and harm under the deployed gate. Report improvement and collateral separately. An optional improvement/collateral ratio is undefined when both are zero and right-unbounded when collateral is zero but improvement is positive; never insert an arbitrary epsilon to turn these cases into a finite claimed advantage.

For a fixed prompt set, retain retrieval features and slot assignments at saved checkpoints. Measure normalized key displacement and the fraction of prompts whose firing slot changes after upstream learning. A repeated edit may legitimately need several prefix slots (one per answer token); count *additional* allocations on repetition, not a presumed single slot for an entire multi-token edit.

D.6 Finite settling, cost, and signal informativeness (P6)

For target-conditioned credit, the default error-coordinate energy is

$$E(e; x, y) = \frac{1}{2} \sum_{l,t} \|e_{l,t}\|^2 + \gamma \text{CE}(f(x; e), y), \quad \gamma = 1.$$

Read this as: the errors e are free variables; the states are reconstructed by running the frozen converted blocks forward with the error variables added in; the first term keeps the errors small and the second term pulls the output $f(x; e)$ toward the target y . Input tokens are fixed. Existing cap retrieval choices are held fixed within a settling call, so that discrete gate switching is not silently introduced into the differentiable energy. Only the current prediction target enters the token-credit call; sequence report-card calls use the declared sum of position losses. If the production representation requires different precision matrices or boundary terms, S0 records the complete energy and uses it consistently in every ePC arm; an energy change is an explicit experiment. The word “clamp” means target conditioning through this declared energy, not setting softmax logits to an infinite one-hot vector.

Initialize errors at zero for each call; no hidden target-conditioned state is carried between examples. Measure E_k after k iterations and the normalized gradient residual

$$r_k = \|\nabla_e E_k\| / \max(1, \|\nabla_e E_0\|).$$

Nominal credit uses eight iterations. Record the smallest iteration attaining 95% of the reduction observed by iteration 64, only when $E_0 > E_{64}$. Call 64 a reference horizon; use “settled” as a convergence claim only if $r_{64} \leq 10^{-3}$ and the terminal changes are small. Otherwise call the arm finite-iteration error credit. A 16-step diagnostic is optional under budget. A functional eight-step method may be evaluated even if it has not converged; that limitation stays in its label and its report.

Report the error-loss Spearman correlation, with constant-vector cases undefined. It measures association, not uncertainty calibration or Bayesian precision. Report actual reverse-mode calls, runtime, and query cost as in Appendix B.

D.7 Continual-learning metrics on immutable tasks

Let $a_{t,i}$ be accuracy on the task at stream position i after completing t tasks in that permutation, and b_i its frozen-base accuracy. Define

$$\text{ACC} = T^{-1} \sum_{i=1}^T a_{T,i}, \quad (5)$$

$$\text{BWT} = (T-1)^{-1} \sum_{i < T} (a_{T,i} - a_{i,i}), \quad (6)$$

$$\text{FWT} = (T-1)^{-1} \sum_{i > 1} (a_{i-1,i} - b_i), \quad (7)$$

$$F = (T-1)^{-1} \sum_{i < T} \left(\max_{i \leq u \leq T} a_{u,i} - a_{T,i} \right). \quad (8)$$

In words: ACC is final average accuracy; BWT is how much earlier tasks changed after later learning (negative means forgetting, positive means later learning helped); FWT is how much a task was already known before it was presented, beyond the frozen base; F is the drop from each task’s best-ever accuracy to its final accuracy. This forgetting measure is nonnegative, so positive backward transfer is recorded by BWT rather than hidden in a negative forgetting value. Report acquisition $a_{i,i}$, not only these aggregates. Report late acquisition gain over the pre-task score, matched by task across orders. A late/early gain ratio is secondary and undefined when its denominator is zero; report both gains.

For immutable evaluation examples x , report the standard deviation of ACC across orders and the average pairwise JS divergence of their output distributions at fixed prefixes. Also report pairwise prediction disagreement and which items are lost. Similar aggregate accuracy can mask different forgotten examples. Use identical probe inputs regardless of a model’s own free-generated prefix. Streams with intentionally superseding corrections are excluded from these immutable-order summaries.

D.8 Sequence editing metrics

For edit e , let Y_e be its valid complete-answer aliases and P_e, N_e its paraphrase and locality prompt sets. With the common greedy-generation policy of E.2, ES (efficacy) is exact normalized complete-answer match to Y_e immediately after editing. GS (generalization) averages this score over held-out paraphrases. RET-ES and RET-GS re-evaluate all valid acquired edits at saved checkpoints and at the end. Report unconditional retained accuracy and survival conditional on immediate acquisition; the latter is undefined if no item was acquired.

Locality LS is agreement with the cap-disabled base’s complete generated answer on N_e ; also report first-token agreement, token-distribution KL at fixed prefixes, and correctness where a reference answer exists. Base agreement is not itself factual truth. Editing filters, tokenization, aliases, answer normalization, and truncations are identical across arms. LM drift is the perplexity ratio and the mean loss difference on a fixed 10^6 -token held-out LM sample. Nats are units of loss difference, not of perplexity.

Checkpoints are at 100, 300, 1,000, and 3,000 edits where within the frozen stream length, plus the endpoint. A cost-limited run reports its exact completed prefix. On the correction challenge, only the currently valid answer counts; report reappearance of the obsolete answer separately. On every stream record edit length, frequency, novelty, and near-neighbour strata so that an aggregate cannot hide which items are retained.

D.9 Actual update-order effects and optional shadow diagnostics

The primary interference measurement is direct. At an immutable checkpoint state s , clone all state, including slot metadata, optimizer/replay state, RNG, caches, and counters. Run $s_{ij} = U_j(U_i(s))$ and $s_{ji} = U_i(U_j(s))$ under the same item-update protocol. Only item order changes; intentional corrections and semantic contradictions are excluded. Use 100 fixed pairs per selected checkpoint, stratified into shared-mechanism, independent-private, and near-neighbour pairs, on independently sampled data. Report

$$D_{ij} = |Q|^{-1} \sum_{x \in Q} \text{JS}(p(x; s_{ij}), p(x; s_{ji})),$$

where Q contains the two edits' fixed evaluation prefixes, paraphrases, and unrelated controls. JS divergence is used because it is symmetric and bounded: with natural logs it lies in $[0, \log 2]$. Also report per-item accuracy changes, update counts, allocations, and evictions. A direct damage matrix uses

$$I_{ij} = \mathbb{E}_{x \in Q_i} [L_x(U_j(U_i(s))) - L_x(U_i(s))],$$

the change in item i 's loss caused by subsequently learning item j . Positive entries indicate harm; negative entries indicate helpful transfer. Compare private and shared strata and both orders. No analytic inverse of a discrete allocation/eviction map is assumed.

Optional local differential analysis. This is the Hessian commutator diagnostic, kept optional and explicitly local. Freeze keys, radii, gates, active slots, and topology at a specified checkpoint. Use a fixed finite block of slot values (or declared LoRA parameters) as θ_W , and $H_i = \nabla_{\theta_W}^2 L_i$. This measures a local surrogate of the update process, not the process itself. Report parameterization and scaling; raw Hessian norms across cap and LoRA coordinates are not intrinsically comparable quantities.

For $C_{ij} = H_i H_j - H_j H_i$ and 16 Rademacher probes v (random vectors with entries ± 1 , for which $\mathbb{E} \|Av\|^2 = \|A\|_F^2$),

$$\widehat{\|C_{ij}\|_F^2} = \frac{1}{16} \sum_v \|H_i H_j v - H_j H_i v\|^2.$$

Each term requires four Hessian-vector products. Normalize by $\|H_i\|_F \|H_j\|_F$ only when this is nonzero; otherwise report raw zero/undefined values. The positive-semidefinite commutator Gram is

$$G_{ab} = \text{Tr}(C_a^\top C_b) \simeq \frac{1}{16} \sum_v (C_a v)^\top (C_b v).$$

For symmetric Hessians C_a is antisymmetric, so this equals $-\text{Tr}(C_a C_b)$, not $\text{Tr}(C_a C_b)$; the sign matters for positive semidefiniteness. For invertible linearized maps $A_i = I - \eta H_i$ at sufficiently small η ,

$$A_j^{-1} A_i^{-1} A_j A_i = I - \eta^2 C_{ij} + O(\eta^3).$$

The singular values of C_{ij} describe this infinitesimal skew generator; they are not exact finite rotation angles of the real learning process. Equation (1) and the scalar counterexample remain essential to interpreting any of this.

For mutually orthogonal coordinate-block projectors P_m with $\sum_m P_m = I$ on the fixed parameter space,

$$\|C\|_F^2 = \sum_{m,n} \|P_m C P_n\|_F^2.$$

Use these full-operator blocks, including cross-bank blocks, for any mass attribution. Separately recomputing each bank's restricted Hessian commutator does not yield this decomposition, because it

drops the cross terms. Include exact small-matrix tests for the Gram sign, the loop sign, and the block accounting before any optional HVP run.

D.10 Router agreement and tracing

On the constructed fixture, $R^*(m, j)$ lists the banks with supplied permission and causal support for latent j . Delivery precision is the fraction of bank choices that are in R^* ; item recall is the fraction of all eligible items receiving at least one such delivery. Abstentions reduce recall. Count proposed deliveries, accepted writes, and successful edits separately. Report per-latent confusion matrices and confidence intervals; majority-class bank assignment (always choose the most common bank) is the baseline against which a router must be judged. For multi-cause items use the set of required mechanisms and report coverage, rather than forcing a single label.

On the learned grammar use controlled corruption and restoration to estimate

$$R_{mj}^{\text{trace}} = \frac{p_{mj}^{\text{restore}}(y) - p_j^{\text{corrupt}}(y)}{p_j^{\text{clean}}(y) - p_j^{\text{corrupt}}(y)},$$

the fraction of the correct-answer probability recovered by restoring the clean residual at bank m 's site. Use pairs with a clean–corrupt probability gap of at least 0.1; report excluded weak pairs and never clip overshoot silently. A recovery of at least 0.5 is a *tracing-supported* site. Keep all qualifying banks, the continuous scores, and ambiguous or no-site cases. Restoring a residual vector may restore information computed upstream of that site, so these sites are intervention evidence, not unique ground-truth mechanism locations, and they are not pooled with constructed-oracle precision/recall.

D.11 Confirmatory comparisons and uncertainty

The primary editing outcome is endpoint RET-GS. For C2 versus each of C1 and CR, report paired differences for every realization/permutation, then means over orders within each realization. The practical target is an RET-GS at least 0.02 higher. A positive primary finding requires that target and a lower uncertainty bound above zero for each required contrast, with immediate ES no more than 0.02 worse and LS no more than 0.01 worse. Report every constraint even if one fails. Resource eligibility for a joint efficiency claim is Appendix B.

Use paired cluster resampling of the three stream realizations, retaining all five committed orders within a realization, with 10,000 bootstrap draws. Report conservative 97.5% two-sided intervals for the two required contrasts, alongside the complete paired table and min/max across orders. These intervals are small-sample descriptions: three stream realizations do not support a strong population claim on their own. Report evaluation-subject resampling separately if used; examples and permutations are not additional independent base or training replicates. If the interval spans meaningful benefit and harm, classify the result as inconclusive. A larger planned independent replication is preferable to extra seeds selected after seeing a near-threshold result. *Reason: the unit of independent variation is the realization; orders within a realization share items, and examples within an order share a model.*

Acquisition/locality noninferiority is supported only when the corresponding lower paired interval stays above its negative margin. A favourable point estimate with unresolved noninferiority is a qualified result. PR-C uses the 0.6/0.8 descriptive bands with intervals; it never defines an implementation pass/fail. Synthetic retained accuracy, damage strata, and order effects are supporting endpoints, with their direction fixed in the manifest. Full causal interpretation requires the oracle fixture, the random control, and shared/private transfer evidence, not a post hoc favourable subset of metrics.

E Stream specifications

E.1 Two synthetic fixtures

MODULAR-CONTROL. Construct a small residual computation graph with three explicit module groups at the candidate write depths. Fixed masks make each private latent influence its designated path and make shared latents influence an explicitly shared path. Final output coordinates are partitioned so that an unrelated site’s permitted write cannot directly overwrite every other site’s output. In this fixture, every arm projects its candidate direction and stored value onto the same fixed allowed write subspace for each bank, before normalizing the direction; the oracle additionally receives the supplied routing labels. These write restrictions do not apply to the learned transformer. Record masks and intervention sites as part of the fixture. This deliberate identifiability is what makes R^* known. Supply at least 100 independent-private, 100 shared, and 100 mixed-cause evaluation items plus held-out combinations. Include a no-sharing case, a useful-sharing case, and a wrong-router case. The oracle control must recover at least 95% of planted deterministic targets with unchanged unrelated outputs within 10^{-6} in probability. This verifies an attainable constructed case, not a universal guarantee for unrestricted residual interventions. *Reason: on a real transformer we do not know where mechanisms live; here we build them in, so that a router’s agreement with the truth can be scored and the measurement code can be checked.*

LATENT-GRAMMAR. Reuse the six-layer grammar transformer with vocabulary 64 and sequence length 64. Define eight observable task contexts, two shared switches, and one private switch per context. Private switches affect only their own contexts; shared switches affect the corresponding mechanism across contexts. The shared switch value is consistent across the immutable stream. A context token or an observable context feature identifies which private rule applies; ambiguous unlabelled task switches are not an acceptable generator.

For the primary mechanism experiment, designate *one* evaluated token position per sequence, with one supplied causal label there, and present one sequence/target pair as the learning item. Other positions may supply context; they are not silently expanded into 64 separate training items. A multi-cause and multi-position extension is a separately labelled challenge. This resolves the original ambiguity between sequence counts and token-update counts.

Generate independent development, training, and evaluation examples by manifest. Use 2,000 evaluation sequences per task and the S4-selected training count. There are three realizations and five balanced task orders; every task appears at least once in the first quarter and once in the last quarter across the five orders. A joint-training reference sees the same total number of training examples. Evaluate held-out combinations of shared/private switches within the base’s declared input grammar. Keep task difficulty and baseline competence in the report.

Causal tracing uses clean/corrupted paired sequences that change one latent while holding the rest fixed. Sample positions and patch depths from a manifest set before observing tracing outcomes. Restore the specified residual sites, use the normalized D.10 score, and retain all multiple-site and no-site cases. Do not redesign latents until a unique preferred depth appears. Tracing on this learned model is a diagnostic, distinct from the constructed R^* .

E.2 Separate sequence-editing streams

ZSRE-3K has a ceiling of 3,000 edits per realization; COUNTERFACT-1K has a ceiling of 1,000. They are separate experiments, not a combined stream. Commit the source release, preprocessing revision, dataset IDs, target strings, aliases, and prompt forms. Use the preprocessed zsRE editing data and CounterFact’s paraphrase/neighbourhood sets [10, 9]. Select only edits for which the BP teacher’s

complete generated answer is outside the accepted target aliases, so that every edit is a real edit. Keep the same selected items for all bases; D.1 reports initial-correctness imbalances. Deduplicate immutable facts and subjects across development and test, and avoid incompatible targets for the same query within a stream.

Each edit supplies a prompt x and canonical answer tokens $y_1, \dots, y_M$, including a fixed terminating newline. Alternatives (aliases) are for scoring unless an explicit common training-augmentation policy is frozen. Tokenize prompt/answer boundaries using one tested helper; assert that the target decodes to the intended answer and delimiter. Exclude and count examples whose canonical answer plus delimiter exceeds 32 tokens *before* any arm is run. Use greedy generation, maximum 32 new tokens, stopping at newline or EOS, for every arm. Score the text before the terminator under a fixed normalization (Unicode normalization, case-folding, whitespace collapse, and trimming of leading and trailing whitespace). Do not strip arbitrary answer content or accept partial-token prefixes. A truncated answer is a failure unless the complete accepted answer was already emitted under the same stopping rule.

Cap training visits the gold prefixes $(x, y_{<t})$ and learns y_t one at a time, using F.3. Evaluation is free generation, never gold-prefix scoring substituted for complete-answer efficacy. Also report teacher-forced NLL, to diagnose acquisition failure. Count every token-prefix update, including the terminator, and compare item costs stratified by target length. *Reason: learning is naturally token by token, but a user sees the whole generated answer; a method must be scored on what a user sees.*

The reference decoder recomputes the full prefix for each next token, with no KV cache, for all compared arms. Cap writes occur only at the current prediction position. A caching optimization may be reported separately only after parity with these semantics is verified, since reusing past cap-modified states can change the model’s computation. Evaluation is read-only and must not update usage, radii, keys, precision heuristics, replay, or RNG-dependent learning state.

Use at least 100 cases each for: near-neighbour prompts requiring different answers; composition of previously learned relations where the reference supports unambiguous evaluation; and explicit temporal corrections. These challenge sets are disjoint from development and reported separately from the primary benchmark. The correction track supplies a stable fact identifier and version to all methods; its latest valid answer replaces the old answer in evaluation. Any method-specific use of revision metadata is disclosed. These intentionally noncommuting revision events are not included in immutable order invariance claims. For the main synthetic and editing runs, the router receives no task, latent, or fact identity that would be unavailable at normal query time; supplied labels are only for oracle fixtures and offline scoring.

LM drift uses a committed 10^6 -token sample from WikiText-103 validation under the common tokenizer, without answer prompts. A smaller model trained only on the synthetic vocabulary cannot be evaluated on this corpus as if it were a GPT-2 language model; record that metric as unsupported.

E.3 Deferred domain stream

Sequential encyclopaedia, code, legal, and biomedical domain adaptation is next month’s work. It is not used to absorb time needed for incomplete core comparisons. A later plan must set data sizes, observability of domain changes, replay permissions, and domain-specific retention metrics.

E.4 Property and regularization data

For domain descriptions, commit disjoint development, regularizer-training, and final-evaluation document manifests, each with eight domains and 256 sequences of length 128. Document-level deduplication is required; different sampling seeds alone do not prove disjointness. If the model’s vocabulary or training does not support these domains, use labelled grammar mechanisms and state

that substitution. Synthetic private/shared labels come from the generator. Domain labels alone do not authorize private/shared separation losses. Development may select a hypothesis; the final evaluation must be unseen by that selection and by S6.

F Cap v0 implementation contract

F.1 Structure and common read/write semantics

Three banks attach at $l_m = \text{round}(mD/3)$, $m = 1, 2, 3$, with distinct valid indices required. The hook is defined precisely as the block output before the next block, and for the final block as the output before the final normalization and head. The same convention is used by the BP and ePC wrappers. At a token-prefix query, p is the last input-token position. Banks act in increasing depth order; earlier writes feed the later blocks. A bank reads before its own write. Only position p is modified by the cap in v0.

Let LN_0 be parameter-free centring and variance normalization with numerical epsilon 10^{-5} , and define $z(h) = \text{LN}_0(h)/\sqrt{d}$, so that keys have unit scale regardless of layer. The default R-h key feature is $q_m = z(h_p^{(l_m)}) \in \mathbb{R}^d$. Keys are frozen after allocation in the primary protocol (the original moving-key EMA is removed); edited upstream activations can still move the query features. Feature normalization and radius calibration are shared by arms using the same base and read variant. *Reason: a key that drifts after allocation makes retrieval failures impossible to attribute; a frozen key makes drift of the query the only moving part, and that is logged.*

A **forward** call chooses, among active slots with $\|q_m - k_s\|_2 \leq \rho_s$, the nearest; ties use the smallest immutable slot ID. A zero radius matches only an exactly equal key. It adds that slot's value v_s , or zero if no slot fires. Evaluation is read-only. For numerical reproducibility, distance and tie rules must not depend on unordered containers.

F.2 Slot state, memory, and conflicts

Each slot contains a key $k_s \in \mathbb{R}^{d_k}$, a value $v_s \in \mathbb{R}^d$, a nonnegative radius, a count of successful training-item uses, creation and last-use indices, the last target-token ID, a fixed-size owner-edit digest, a version, and an optional pre-update-loss EMA. Store no unbounded text in metadata. Count each training item at most once per slot even if several rounds touch it. Inference and offline evaluation do not increase the training-use count. Caches and indices are included in the total persistent memory measurement.

All mutations are transactional: a candidate change is applied, evaluated, and either committed or rolled back completely. When the firing slot's last target token conflicts with the presented token, resolve the conflict *before* changing its value or key. For distinct keys at distance $d_{qs} > 0$, shrink the old radius to $\min(\rho_s, 0.49d_{qs})$ and initialize the new slot's radius to $\min(\rho_0, 0.49d_{qs})$. Leave the old value and key intact. The factor 0.49 avoids a shared midpoint without allowing a negative radius. Recompute actual retrieval for every candidate; overlap with another slot may still affect efficacy.

For identical keys with incompatible targets, no pair of radius balls can separate them. If the event is an explicit newer revision of the same fact in the correction track, retire the slots carrying that older fact/version digest and attempt the replacement; this metadata-assisted operation is labelled and available only on that track. Otherwise preserve the old memory, log **ambiguous-key-conflict**, and reject this candidate bank; the item may try its other scheduled banks and rounds. Never claim that both incompatible targets were stored successfully. Exact revision replay is idempotent.

If a bank is full, evict the slot with the lowest successful-use count, breaking ties by oldest last-use and then smallest ID; charge the eviction and retain its reason. Eviction and conflict edits commit

only together with a successful candidate write. A rejected candidate restores values, metadata, radii, allocation IDs, and RNG state exactly. The bounded correction index stores only active digests and is included in memory accounting.

F.3 Learning a complete edit

For an immutable edit $(x, y_{1:M})$, visit each gold prefix in order. The item budget covers every prefix; editing a long answer is not counted as one token update. At each prefix run the following for at most $R = 5$ rounds, stopping when $\text{CE}(p, y_t) \leq \tau_{\text{edit}} = 0.1$ nats. End-of-item ES is measured by free generation. Failure to meet the threshold is recorded as acquisition failure, not raised as an exception.

1. **Current prediction.** Run the current cap and compute $L = \text{CE}(p, y_t)$ and $e_o = p - \text{onehot}(y_t)$. This is the residual error after the present memory contribution. It is not a decomposition into shared and private residuals, nor a separate statistical novelty estimate. The primary cap uses the full output error; top-32 token masking is deferred.
2. **Candidate descent directions.** For adjoint credit, $g_m = \partial L / \partial h_p^{(l_m)}$ and $d_m = -g_m / \|g_m\|$. The discrete current retrieval choices are held fixed for differentiation. For a common output loss one reverse pass can expose all requested layers; do not artificially charge one backward pass per bank if the implementation reuses that pass. For ePC credit use the F.6/D.6 finite-inference error field at the corresponding site and the wrapper’s documented descent sign, then normalize. The sign convention is verified once on an analytic differentiable control in S0; it is never flipped per test example to obtain a desired result. A zero direction produces an explicit no-direction outcome; arbitrary directions are not inserted.
3. **Measured bank selection (C2).** Make a temporary forced additive write $\epsilon b_m d_m$, where b_m is the development median raw residual norm at that bank and $\epsilon = 0.01$. Recompute the downstream forward pass, including actual discrete retrieval, without committing memory state. The score is signed:

$$r_m = \frac{L - L_m^{\text{probe}}}{\epsilon}.$$

Select the bank with the largest positive score, ties by depth, only when the loss reduction exceeds $\max(10^{-8}, 10^{-6}L)$; otherwise abstain for that round. These are direction-specific intervention scores, not causal-source labels. Probe rejection and the actual write may differ because allocation and routing change; both are logged. Probe-size sensitivity at 0.003 and 0.03 is an optional development diagnostic, not extra per-test selection.

4. **Other delivery rules.** C0 schedules the last bank; C1 schedules all three; CR schedules one from its fixed development bank distribution; CO schedules the supplied permitted bank(s) in depth order. CR randomness is keyed by seed, item ID, prefix index, and round, which allows common random choices for an item in reversed-order probes without reading future labels. Zero-direction and conflict cases are logged for every rule. The common nominal aggregate normalized write budget is A per round.
5. **Transactional update search.** If b banks are scheduled, reserve A/b for each, with no redistribution of unused shares. Process them in increasing depth order. Recompute current features, gates, loss, and credit after each accepted earlier-bank write. Resolve the candidate bank’s conflict or allocation in a private transaction as F.2 specifies, preserving the pre-transaction state as the no-op option. Candidate incremental values use

$$\Delta v_s = w_s a b_m d_m, \quad a \in \{A/b, A/(2b), A/(4b), A/(8b)\}.$$

Here $w_s = 1$ in the primary protocol; any optional difficulty weight is already included in the tested update. For a new slot initialize its value to zero before the candidate increment. Test

candidates from the same original transaction snapshot, not cumulatively. Evaluate the actual resulting cap with all ordinary retrieval rules. Choose the candidate with the smallest loss, breaking ties by the smaller normalized increment; commit it only if the loss improves by more than $\max(10^{-8}, 10^{-6}L)$. Otherwise restore the complete original state. There is no assumption of monotonic loss along a direction and no requirement that a fixed 90% reduction be reachable. *Reason: a geometric candidate grid with a no-op option is a bounded line search that cannot loop, cannot assume convexity, and always has a safe outcome.*

6. **Budget accounting.** The sum of normalized accepted value increments $\sum_m \|\Delta v_m\|/b_m$ is at most A per round. This is a cap parameter-write budget; radius changes, evictions, and gate switches can produce additional functional changes, which are measured separately. Rejected searches and routing probes consume compute. A forced probe is never retained as an unlogged learned write. The primary search uses at most four forward candidates per scheduled bank per round; all call counts are recorded.
7. **Metadata.** After commit, update the target token, owner digest, version, use count, and last-use index consistently. The optional loss EMA uses the *pre-update* loss of the item that actually used the slot, with rate 0.2. Never initialize it from a conveniently small fitted residual and call it Bayesian precision. Repeat from a fresh forward pass, or stop at the threshold or at the round or resource limit.

The optional difficulty arm sets $w_s = \max(0.1, 1/(1 + \bar{L}_s/L_{\text{ref}}))$, with $L_{\text{ref}} > 0$ fixed on development data and $\bar{L}_s$ the prior pre-update-loss EMA; new slots use $w_s = 1$. This is a heuristic sensitivity analysis, not a variance estimate. Every committed candidate already includes w_s in its measured loss.

F.4 Read variants and query-time features

R-h.

$q_m = z(h_m)$ from the current edited pass; $d_k = d$. The default.

R-h0.

$q_m = z(h_m^0)$ from a separate cap-disabled pass on the same prefix; writes still act in the edited pass; $d_k = d$. This isolates retrieval movement caused by upstream cap changes, and charges the extra forward pass.

R-g.

From that same cap-disabled pass, set $\hat{y} = \arg \max p_0$ and compute the adjoint g_m^0 of $\text{CE}(p_0, \hat{y})$. Use $q_m = [z(h_m^0); z(g_m^0)]/\sqrt{2}$, $d_k = 2d$. Available on BP and ePC. This is the same-dimensional, label-free control for R-e.

R-e.

On ePC, use cap-disabled target-conditioned error inference with that same label-free $\hat{y}$ and the query budget, and form $q_m = [z(h_m^0); z(e_m^0)]/\sqrt{2}$, $d_k = 2d$.

Use the same label-free key computation during learning and inference; true labels enter credit computation only during learning. R-e and R-g receive matched byte ceilings and dimension, with separate development calibration of their radii. Report R-g on ePC as the same-base control for R-e; BP R-g tests whether the added-feature idea also helps BP. Any zero auxiliary vector is recorded and normalized by the declared epsilon, not replaced by true-target information. These keys are functions of the cap-disabled pass, which avoids a circular dependence on which memory the key itself retrieves. No target-conditioned transient state is cached across unrelated queries. *Reason: a key that used the true answer would be a key that only exists when you already know the answer; the pseudo-label $\hat{y}$ is what the model can compute at query time.*

F.5 Positive controls and failure semantics

PC-1: planted acquisition.

On the constructed fixture, the supplied router and known attainable edits recover at least 19 of 20 targets within the round budget. Supply the exact support for the invariant portion of this test. Unrelated outputs remain unchanged to the declared numerical tolerance.

PC-2: support and cap-off identity.

An oracle gate that is false on unrelated inputs produces exactly their cap-disabled computation. With the whole cap disabled, outputs match the base reference on 256 probes. A checksum alone is not a behavioural identity test.

PC-3: repeat idempotence.

For 20 edits that reached the per-prefix threshold, with no intervening updates, repeating the same complete edit allocates no additional slots. It need not have had only one slot initially.

PC-4: conflict transaction.

On hand-built distinct-key conflicts, the old value and key survive, radii remain nonnegative, and failed replacements roll back every change. Test identical-key ambiguity, a supplied newer version, and repeated delivery of that same version separately.

PC-5: signed probe and search.

On an analytic one-dimensional loss, a helpful probe has a positive score and a harmful probe a negative score. A nonmonotone test loss does not crash and is not assumed to bracket a root. Rejection restores the exact learner state. The accepted update is precisely the one evaluated, including any difficulty weight.

PC-6: memory and write budgets.

C0/C1/C2/CR respect the same total byte ceiling; R-e/R-g keys reduce the allowable slot count as required. Verify aggregate normalized increments and deterministic eviction at full capacity.

PC-7: complete answers.

A two-token answer with the correct first and incorrect second token fails ES. Teacher-forced success does not override free-generation failure. Terminators and truncation follow the common policy.

PC-8: label isolation.

Query features receive only the input prefix and the base-derived pseudo-label. Changing a hidden evaluation answer while holding the prefix fixed cannot change R-e/R-g keys or the prediction. All evaluation is read-only and leaves the complete learner state unchanged.

PC-9: cloning and randomness.

Cloned identical update sequences agree; reversed sequences begin from the same complete state and use the declared item-keyed random choices. The scalar quadratic order effect is recovered.

PC-10: reference parity.

Validate GRACE and other baseline adaptations on small shared single- and multi-token cases, documenting intentional changes such as memory eviction. Baseline underperformance is not a parity test.

Router accuracy on the learned grammar, superiority over LoRA, and any particular PCA pattern are scientific measurements. They are deliberately absent from this list of implementation invariants.

F.6 Defaults and frozen configuration

Quantity	Default	Calibration or interpretation
Banks	$D/3, 2D/3, D$	Rounded, distinct, explicit hook locations
Memory	$6144(8d + 128)$ bytes	Actual dimensions and metadata determine slots
Key radius	S2 calibrated	Per layer and read variant; shared within a contrast
Residual scale b_m	Median raw norm	Cap-disabled development prefixes; floor at 10^{-8}
Aggregate step A	0.1	S2 candidates 0.03, 0.1, 0.3
Probe ϵ	0.01	Signed improvement; all probes charged
Round limit R	5 per prefix	Complete edit cost sums all target prefixes
Search trials	4 per bank	Geometric candidates plus no-op
Stop loss	0.1 nats	Token CE; free-generation efficacy reported separately
Key motion	None after allocation	Retrieval features may still move under upstream writes
Difficulty weight	Off	Optional explicit heuristic, not precision
Settling horizon	8	Exact energy, solver, and learning rate recorded from S0
Realizations/orders	3×5	Data and order variation, paired across arms

The production ePC solver, step size, precision conventions, state reset, and stopping behaviour must be filled in from the actual implementation in S0. A bare instruction to run eight unspecified “settle steps” is not executable. Solver changes are development choices with a logged comparison, never an unreported per-arm fallback. D.6 defines the default energy when compatible with the production architecture. All declared numerics are frozen at S4.

F.7 Package interface

The package `pccap` supplies `BPBase`, `EPCBase`, `Bank`, `Cap`, `Router`, `Transport`, the fixtures, and `Harness`. Base wrappers expose forward, hidden hooks, adjoint, finite error inference, checksums, and a per-call cost record. Cap exposes read-only predict, transactional update, serialize, restore, clone, and memory-bytes. Router variants are `Last`, `Full`, `Measured`, `Random`, and `Supplied`. Dataset manifests separate development and test. All configuration paths and hashes are explicit.

```
pccap run --stage S4 --arm C2 --base BP --read h
--realization 0 --perm 0 --manifest frozen.json
```

The command fails if a required frozen field is missing. Decision records carry item digest, prefix and round, candidate banks, signed scores, chosen route, accepted increment, before/after loss, allocation/conflict/eviction codes, and all cost counters. They contain no undisclosed evaluation answers in inference features. Human-readable logs and machine-readable outcomes agree.

G Report template

Every stage report contains:

1. Header: stage, source/configuration/checkpoint hashes, hardware, realizations and orders, elapsed cost against its ceiling.
2. Status: development or confirmation; completed, partial, correctness failure, resource limitation, or scientifically negative/inconclusive.
3. Controls: expected invariant, observed value, and pass/fail; geometry alerts and scientific outcomes in separate tables.
4. Coverage: all planned arms and completed prefixes; exclusions, ambiguous items, weak tracing pairs, unsupported and undefined measurements.

5. Results: acquisition, retained ES/GS, locality, LM drift, transfer, order effects, memory and cost; complete paired tables and uncertainty.
6. Mechanism evidence: oracle and control results, random-routing comparison, signed intervention scores, shared/private effects, key drift, and evictions.
7. Optional mathematics: frozen topology and coordinates and HVP controls if those diagnostics were run; limitations relative to actual updates.
8. Deviations: each change, its evidence, its timing relative to test access, and whether both sides of affected comparisons were rerun.
9. Interpretation: claims supported by each controlled contrast, claims not identified by it, and plausible explanations of negative results.
10. Reproduction: commands, manifest paths, licences and provenance, and remaining work in priority order. Include surprises without retrospectively changing the primary outcome.

H Revision map and interpretation boundaries

This map records how the technical review’s findings on the original plan were resolved, so that a reader of the original knows what changed and why.

Issue in the original plan	Change in this revision
Commuting Hessians treated as an exact order criterion	Actual complete-state update reversals; correct gradient bracket and counterexample; scoped optional Hessian analysis
Negative participation penalty	Positive concentration penalty plus dataset coverage; zero-field failure handling
Domain overlap treated as inseparable interference	Private/shared intervention and transfer checks; domain PCA stays descriptive
Probe magnitude treated as causal responsibility	Signed intervention improvement; constructed oracle versus distributed tracing
One-bank versus three-bank capacity confound	Equal total bytes, C0 depth control, CR sparse random control, aggregate write budget
Different routing rules in BP/ePC arms	Common bank-level routing; representation, credit, and key contrasts separated
Settling described as requiring no backward computation	Explicit reverse-mode cost and declared finite-inference energy/solver
Cap called PC without its own inference process	Precise PC-motivated v0 scope; full cap-level PC remains a later design
Conflict, radius, key, and search ambiguities	Transactional pre-write conflict handling, nonnegative radii, fixed keys, bounded candidate search
One-token editing and ambiguous stream count	Complete-answer autoregressive protocol; separate ZSRE-3K and COUNTERFACT-1K streams
Aggregate order scores and invalid Gram/sign conventions	Per-example divergence and damage; PSD Gram, corrected loop sign, full block accounting
Rigid untested numerics and oversized grid	Bounded disjoint development, preregistered core, throughput-based scope, one integrated report

The earlier internal reports remain references rather than newly audited evidence. All claims about the production checkpoints are conditional on S0/S1 reproduction. The programme is successful as a research month when it produces a valid answer with a clear tested scope, including an informative negative answer; it does not promise a routing or ePC advantage in advance.

References

- [1] GPT5.6-Pro and B. Goertzel, “Causally Designed Error Highways: Long-Range Credit Transport Without Diffuse Task Entanglement,” working note, July 2026.
- [2] ZeroBot, ProtoMegaBot and B. Goertzel, “A Multiscale, Laterally Connected Predictive-Coding Cap for Continual Learning on ePC Transformers,” design document, July 2026.
- [3] B. Goertzel and Claude Fable, “The Coupling Dilemma: Reach versus Conditioning, Sharing versus Forgetting, and Catastrophic Forgetting as Curvature of the Task Manifold,” working draft, 2026.
- [4] B. Goertzel, “The Shadow Calculus: Which Curriculum Interference Is Observable?” working note, 2026.
- [5] B. Goertzel and collaborators, “Causal Fibres for Partial Transformer Distillation,” synthesis report and programme document v1.1, July 2026.
- [6] C. Goemaere, G. Oliviers, R. Bogacz and T. Demeester, “ePC: Fast and Deep Predictive Coding in Digital Simulation,” arXiv:2505.20137v5, 8 June 2026. Earlier versions titled “Error Optimization: Overcoming Exponential Signal Decay in Deep Predictive Coding Networks.” <https://arxiv.org/html/2505.20137v5>.
- [7] T. Hartvigsen, S. Sankaranarayanan, H. Palangi, Y. Kim and M. Ghassemi, “Aging with GRACE: Lifelong Model Editing with Discrete Key-Value Adaptors,” NeurIPS 2023. Official implementation: <https://github.com/Thartvigsen/GRACE>.
- [8] P. Wang et al., “WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models,” NeurIPS 2024. <https://arxiv.org/html/2405.14768v3>.
- [9] K. Meng, D. Bau, A. Andonian and Y. Belinkov, “Locating and Editing Factual Associations in GPT,” NeurIPS 2022.
- [10] O. Levy, M. Seo, E. Choi and L. Zettlemoyer, “Zero-Shot Relation Extraction via Reading Comprehension,” CoNLL 2017.
- [11] E. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR 2022.
- [12] J. Kirkpatrick et al., “Overcoming Catastrophic Forgetting in Neural Networks,” PNAS 114(13), 2017.
- [13] S. Dohare et al., “Loss of Plasticity in Deep Continual Learning,” Nature 632, 2024.